

Section I: Data Structures [15 Marks]

Q1) Read the data from the 'employee.txt' file and sort on age using Merge sort. Write the sorted data to another file 'sortedemponage.txt'. Assumed file format (one record per line): Name Age e.g. Raj 30

Answer:

```
#include <stdio.h>

struct Emp
{
    char name[50];
    int age;
};

void merge(struct Emp a[], int l, int m, int r)
{
    struct Emp b[100];
    int i = l, j = m + 1, k = 0;

    while(i <= m && j <= r)
    {
        if(a[i].age <= a[j].age)
            b[k++] = a[i++];
        else
            b[k++] = a[j++];
    }

    while(i <= m)
        b[k++] = a[i++];
    while(j <= r)
        b[k++] = a[j++];

    for(i = l, k = 0; i <= r; i++, k++)
        a[i] = b[k];
}

void mergesort(struct Emp a[], int l, int r)
{
    int m;

    if(l < r)
    {
```

```

        m = (l + r) / 2;
mergesort(a, l, m);
mergesort(a, m + 1, r);
merge(a, l, m, r);
    }
}

int main()
{
    FILE *fp, *out;
    struct Emp e[100];    int n
    = 0, i;

    fp = fopen("employee.txt", "r");
    if(fp == NULL)
    {
        printf("File not found");
        return 0;
    }

    while(n < 100 && fscanf(fp, "%s %d", e[n].name, &e[n].age) == 2)
n++;    fclose(fp);

    if(n == 0)
    {
        printf("No records");
        return 0;
    }

    mergesort(e, 0, n - 1);

    out = fopen("sortedemponage.txt", "w");
    if(out == NULL)
    {
        printf("Cannot create output file");
        return 0;
    }

    printf("Sorted data:\n");
    for(i = 0; i < n; i++)
    {
        fprintf(out, "%s %d\n", e[i].name, e[i].age);
        printf("%s %d\n", e[i].name, e[i].age);
    }
    fclose(out);

    return 0;
}

```

Section II: Database Management Systems [15 Marks]

Q 2) Consider the following database (Primary keys are underlined)

Customer (cust_no, cust_name, cust_city)

Account (acc_no, acc_type , balance)

Account type can be either “Saving” or “Current”

There exists a One-to-Many relationship between Customer and Account.

Q 2(A)

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

Answer:

```
CREATE TABLE Customer(  
  cust_no INT PRIMARY KEY,  
  cust_name VARCHAR(50),  
  cust_city VARCHAR(50)  
);  
  
CREATE TABLE Account(  
  acc_no INT PRIMARY KEY,  
  acc_type VARCHAR(10)  
    CHECK(acc_type IN ('Saving','Current')),  
  balance NUMERIC(14,2),  
  cust_no INT REFERENCES Customer(cust_no)  
);
```

```
INSERT INTO Customer VALUES  
(1,'Amit','Delhi'),  
(2,'Neha','Ahilyanagar'), (3,'Raj','Pune');
```

```
INSERT INTO Account VALUES  
(101123,'Saving',50000,1),  
(101124,'Current',2000000,3),  
(101125,'Saving',70000,2);
```

Q 2(B)

B .Execute the following queries in PostGreSQL (any 3) [6 marks]

i. Find the names of all Customers staying in “Delhi”.

Answer:

```
SELECT cust_name FROM Customer WHERE  
cust_city = 'Delhi';
```

ii. Give the total balance of all Saving accounts.

Answer:

```
SELECT SUM(balance) FROM Account WHERE  
acc_type = 'Saving';
```

iii. Find the maximum balance of an account.

Answer:

```
SELECT MAX(balance) FROM Account;
```

iv. Increase the balance of account no 101123 by Rs 10,000/-

Q 2(C)

C. Write a query to find the names and phone numbers of all Customers of Current accounts, having balance greater than 10,00,000/-

Answer:

```
-- Customer table has no phone column, so add it first  
ALTER TABLE Customer  
ADD phone_no VARCHAR(15);
```

```
SELECT DISTINCT C.cust_name, C.phone_no  
FROM Customer C  
JOIN Account A ON C.cust_no = A.cust_no  
WHERE A.acc_type = 'Current'  
AND A.balance > 1000000;
```

Q 3) Viva [5 marks]